LMAT1351 Approximation : méthodes et théorie
Projet 2023

Maximilien Forte et Tom Claeys

Consignes générales

Date limite. Le projet doit être rendu au plus tard le 22 décembre 2023 à 23h59. Les projets reçus après cette date ne seront pas corrigés, sauf dans des cas exceptionnels (cas démontré de force majeure, attestation médicale ...). Vous enverrez votre projet au format pdf par e-mail à `tom.claeys@uclouvain.be` et à `maximilien.forte@uclouvain.be`.

Réalisation et rédaction. On vous demande de répondre aux questions posées d'une manière détaillée et argumentée, de construire les algorithmes demandés, et de rendre un rapport correct, lisible, compréhensible, cohérent et bien présenté, avec des illustrations informatives et pertinentes. Le projet doit contenir les réponses aux questions, ainsi que vos codes et vos illustrations. Vous ne devez pas répondre point par point à chaque question dans l'ordre proposé. On vous demande plutôt de rédiger un rapport cohérent pour les 5 sections qui contient les éléments de réponses à chaque question, mais dans l'ordre de votre choix et de la manière que vous jugez informative et pédagogique.

Afin de réaliser des parties du projet, il peut être utile de définir, outre les fonctions demandées explicitement, des fonctions auxiliaires. Dans ce cas, il faut les inclure dans le rapport.

Afin de réaliser votre projet, vous pouvez travailler individuellement ou en binôme. Si vous travaillez à deux, il est obligatoire de rajouter au début du projet un petit paragraphe pour expliquer comment vous vous êtes organisés à deux et qui a contribué plus dans quelle partie du travail. Les étudiant.e.s qui travaillent individuellement peuvent se dispenser des questions 1.5, 2, 3.5 et 4.6–4.8.

Langage de programmation. Vous pouvez utiliser le langage de programmation de votre choix, tant qu'il peut résoudre un système linéaire et faire des graphiques. Nous conseillons cependant un langage qui possède déjà les fonctions que vous devrez écrire, pour pouvoir comparer et vérifier vos résultats. Nous recommandons le module mpmath (mpmath.org) pour python, qui est très intuitif pour les mathématiciens et dont la documentation (en anglais) est très bien expliquée.
Pour la rédaction du projet, il est fortement conseillé d'utiliser Latex.

Evaluation. L'évaluation du projet portera sur plusieurs aspects : explications et argumentations mathématiques, fonctionnement des algorithmes, pertinence des illustrations, cohérence du projet, etc. L'optimisation générale de vos algorithmes (mémoire, calculs inutiles, pratiques instables) sera appréciée, mais jouera dans l'évaluation dans une moindre mesure.

Utilisation des ressources. Il n'est pas interdit de chercher de l'aide sur internet, dans des livres, ou d'utiliser d'autres ressources qui peuvent vous aider pour construire vos codes. Il va de soi qu'il est obligatoire d'indiquer rigoureusement les ressources utilisées (par exemple, dans une bibliographie). Si vous utilisez des outils d'intelligence artificielle (ChatGPT, Bing ...), il est obligatoire de le mentionner, et d'expliquer en détail dans votre rapport de quelle manière vous l'avez utilisé.

1 Interpolation polynomiale

1. Définir une fonction qui renvoie la valeur du polynôme d'interpolation construit à partir des noeuds et des valeurs d'interpolation.

2. Calculer le polynôme d'interpolation de la fonction exponentielle avec des noeuds d'interpolation équi-distants dans $[-1, 1]$, pour différentes valeurs du degré n. Dessiner le graphe de l'exponentielle ainsi que les graphes de différents polynômes d'interpolation dans une même figure. Observer et discuter vos résultats. Comparer la qualité de l'approximation avec l'erreur théorique obtenue via le théorème du reste de Lagrange.

3. Même question, mais maintenant pour la fonction $f(x) = \frac{1}{1+25x^2}$ au lieu de la fonction exponentielle.

4. Même question, mais maintenant pour la fonction $f(x) = |x|$.

5. Est-ce que les polynômes d'interpolation forment une bonne approximation dans les trois cas ci-dessus? Si ce n'est pas le cas, essayer de modifier le choix de noeuds afin d'améliorer l'approximation. Discuter et illustrer.

2 Courbes de Bézier

Ecrire une fonction qui renvoie la courbe de Bézier associée aux noeuds $P_0, \ldots, P_1$. Illustrer le fonctionnement de votre fonction avec quelques figures qui montrent la courbe ainsi que les noeuds.

3 Splines d'interpolation

1. Définir une fonction *splineinterpol* qui renvoie la valeur de la spline cubique naturelle d'interpolation, construite à partir des noeuds *nodes* et des valeurs d'interpolation *values*.

2. Calculer la spline cubique naturelle d'interpolation de la fonction exponentielle avec des noeuds d'interpolation équi-distants dans $[-1, 1]$, pour différentes valeurs du degré n. Dessiner le graphe de l'exponentielle ainsi que les graphes de différents polynômes d'interpolation dans une même figure. Observer et discuter vos résultats.

3. Même question, mais maintenant pour la fonction $f(x) = \frac{1}{1+25x^2}$ au lieu de la fonction exponentielle.

4. Même question, mais maintenant pour la fonction $f(x) = |x|$.

5. Comparer la qualité d'approximation dans les trois exemples ci-dessus avec celle des polynômes d'interpolation.

4 Polynômes orthogonaux

1. Définir une fonction qui renvoie les coefficients de récurrence des polynômes de Legendre.

2. Définir une fonction qui renvoie la valeur du polynôme orthonormal p_n de degré n associé aux suites (ou fonctions définies sur $\mathbb{N}$) $(a_k)_{k\in\mathbb{N}}$ et $(b_k)_{k\in\mathbb{N}}$ construites par les coefficients de récurrence.

3. Dessiner les graphes de polynômes de Legendre de plusieurs degrés.

4. Calculer et dessiner les racines des polynômes de Legendre de plusieurs (grands) degrés. Vous pouvez utiliser une fonction prédéfinie comme *polyroots* de *mpmath* pour calculer les racines d'un polynôme.

5. Vérifier et illustrer la propriété d'entrelacement pour les polynômes de Legendre. Illustrer la distribution des racines des polynômes via des histogrammes. Observer et discuter les résultats.

6. Dessiner les graphes de polynômes de Hermite de plusieurs degrés.

7. Calculer et dessiner les racines des polynômes de Hermite de plusieurs (grands) degrés. Vous pouvez utiliser une fonction prédéfinie comme *polyroots* de *mpmath* pour calculer les racines d'un polynôme.

8. Vérifier et illustrer la propriété d'entrelacement pour les polynômes de Hermite. Illustrer la distribution des racines des polynômes via des histogrammes. Observer et discuter les résultats.

5 Intégration par quadrature Gaussienne

1. Définir une fonction qui renvoie les poids de Christoffel $\{A_{n,k}\}_{k=0:n}$ associés aux coefficients de récurrence.

2. Calculer les poids de Christoffel pour le poids de Legendre.

3. Définir une fonction qui renvoie la valeur approchée de l'intégrale de la fonction f sur l'intervalle $[-1, 1]$ en utilisant la quadrature de Gauss-Legendre.

4. Pour différentes valeurs du nombre de noeuds, tester la précision et commenter vos résultats avec les fonctions suivantes sur $[-1, 1]$:

$$\frac{1}{1+25x^2}, \qquad |x|, \qquad e^x.$$

5. Choisir un nombre de noeuds afin d'obtenir une erreur absolue inférieure à 10^{-3} sur les exemples précédents.